Hands-On Generative AI

Large Language Models

Day 2 - Session 2

KavinSchool

What We'll Cover

1. Text generation before pre-trained LLMs
2. BERT (encoder) vs GPT (decoder)
3. The Transformer at a high level
4. Demo: GPT generation
5. Demo: A Hugging Face tour
6. Demo: BERT contextual embeddings
7. Lab 2.2: introduction to BERT and GPT
8. The modern LLM landscape

Before Pre-trained LLMs

How we used to generate and process text

KavinSchool

The Old Approach

Before pre-trained models, each task started almost from scratch:

- Build features by hand for every task
- Train a separate model per task, per language
- Small labeled datasets limited how well models generalized

Recurrent neural networks (RNNs) read text one word at a time, which was slow and forgot long-range context.

The Shift: Pre-training

Pre-trained models changed the workflow:

- Train once on huge unlabeled text to learn language broadly
- Reuse that model for many tasks with little or no extra data
- Fine-tune or prompt instead of building from zero

This is transfer learning for language, the idea Day 3 builds on.

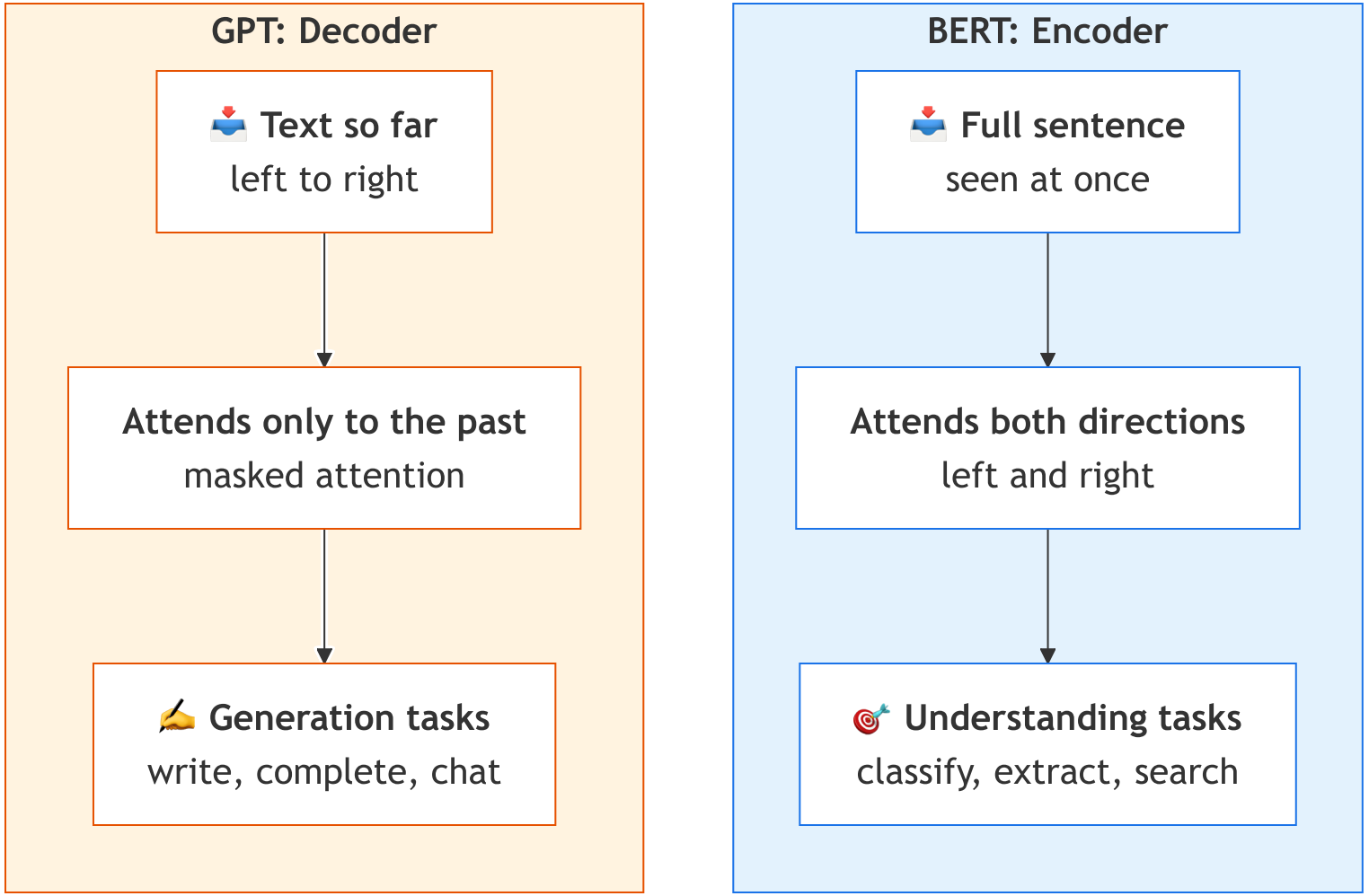
> **IMPORTANT**
>
> You almost never train an LLM from scratch. The modern workflow is adapting a pre-trained model to your specific domain via fine-tuning, RAG, or prompting.

Two Families: BERT and GPT

Encoders and decoders

KavinSchool

Encoder vs Decoder



BERT: The Encoder

BERT (bidirectional encoder representations from transformers) reads the whole sentence at once and attends in both directions.

- Trained by masking words and predicting them
- Builds a rich understanding of each token in context
- Best for understanding tasks: classification, extraction, search

BERT does not generate text one word at a time.

GPT: The Decoder

GPT (generative pre-trained transformer) reads left to right and predicts the next token.

- Trained to continue text, one token at a time
- Attends only to tokens that came before (masked attention)
- Best for generation: writing, completion, chat

Every modern chat model is a decoder at heart.

Same Building Block

Both are built from the Transformer. The difference is how attention is allowed to look:

	BERT	GPT
Direction	Both ways	Left to right
Trained to	Fill in blanks	Predict next token
Shines at	Understanding	Generation

Intuit Scenario: Credit Karma Text Processing

Different tasks call for different architectures.

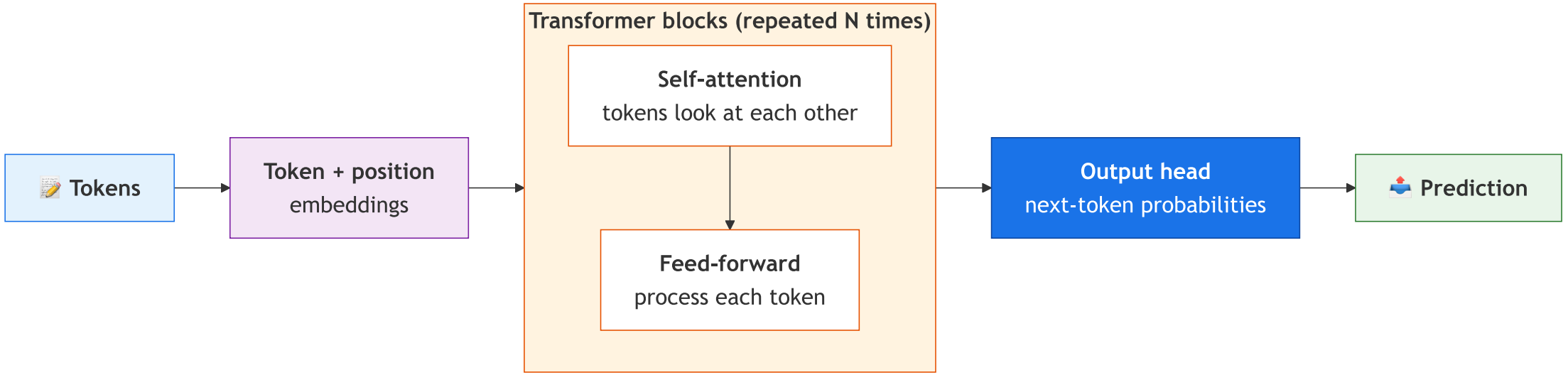
- **The challenge:** Extracting features from credit dispute letters versus drafting a response.
- **Understanding:** An encoder (like BERT) scans the letter to classify the dispute type and extract amounts.
- **Generation:** A decoder (like GPT) takes the extracted facts and drafts a polite, coherent response to the user.

The Transformer

The architecture behind both

KavinSchool

The Transformer at a High Level



Self-Attention

Attention lets each token look at every other token and decide what matters.

- "it" in a sentence can attend to the noun it refers to
- Long-range links are direct, not passed word by word
- All tokens are processed together, which is fast on GPUs (Graphics Processing Units)

Attention is the key idea that made large models practical.

In Practice: When translating "The bank of the river" vs "The bank on the corner," attention lets the model use the surrounding words to pick the right meaning for "bank."

Why the Transformer Won

- **Parallel:** processes a whole sequence at once, unlike RNNs
- **Scales:** more data and parameters keep improving results
- **General:** the same block works for text, code, images, and audio

This scalability is what made today's large models possible.

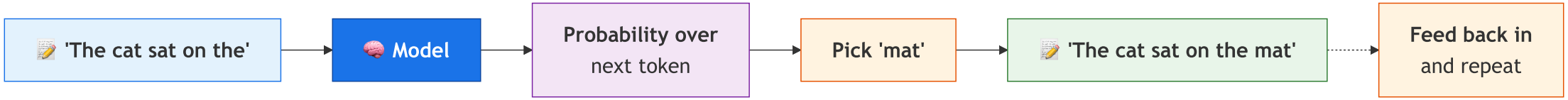
> **NOTE**
>
> The core Transformer architecture from 2017 is largely unchanged in today's frontier models. The massive gains come from scaling up data and compute.

Demo

Seeing pre-trained models in action

KavinSchool

GPT Predicts the Next Token



The model outputs a probability for every possible next token, picks one, appends it, and repeats.

Demo: GPT Generation

Run `day2/demos/demo-autoregressive.py`.

- Feed a prompt to a small GPT-2 model
- Print the top candidate next tokens with their probabilities
- Generate a few tokens and watch the text grow

> Generation is repeated next-token prediction. Nothing more mysterious than that.

A Tour of Hugging Face

Hugging Face is the hub for open models and datasets.

- **Models:** thousands of pre-trained models to download and run
- **Datasets:** ready-to-use data for training and evaluation
- **Spaces:** live demos you can try in the browser

The `transformers` library loads any of these models in a few lines.

The `pipeline` Helper

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
print(classifier("I love this course!"))
# [{'label': 'POSITIVE', 'score': 0.99}]
```

`pipeline` handles tokenizing, running the model, and decoding the output.

Demo: A Hugging Face Tour

Run `day2/demos/demo-huggingface-tour.py`.

- Run three tasks with one API (Application Programming Interface): sentiment, generation, fill-mask
- Watch the `pipeline` helper tokenize, run, and decode
- See how pre-trained models are downloaded and cached

> One API, many tasks: the pipeline hides the complexity of running a model.

Demo: BERT Contextual Embeddings

Run `day2/demos/demo-contextual-embeddings.py`.

- Run the word "bank" through BERT in a river sentence and two money sentences
- Pull the vector for that exact token in each sentence
- Compare with cosine similarity: the two money vectors sit close, the river one apart

> Runs locally, no API key. Same word, different vectors: context shapes meaning.

Breakout Lab 2.2

Introduction to BERT and GPT

KavinSchool

Lab 2.2: BERT and GPT

Open `day2/lab2_2_bert_gpt/start.ipynb` in Colab.

- Use BERT to fill in a masked word (understanding)
- Use GPT-2 to continue a prompt (generation)
- Inspect tokens and next-token probabilities
- See why each architecture fits its tasks

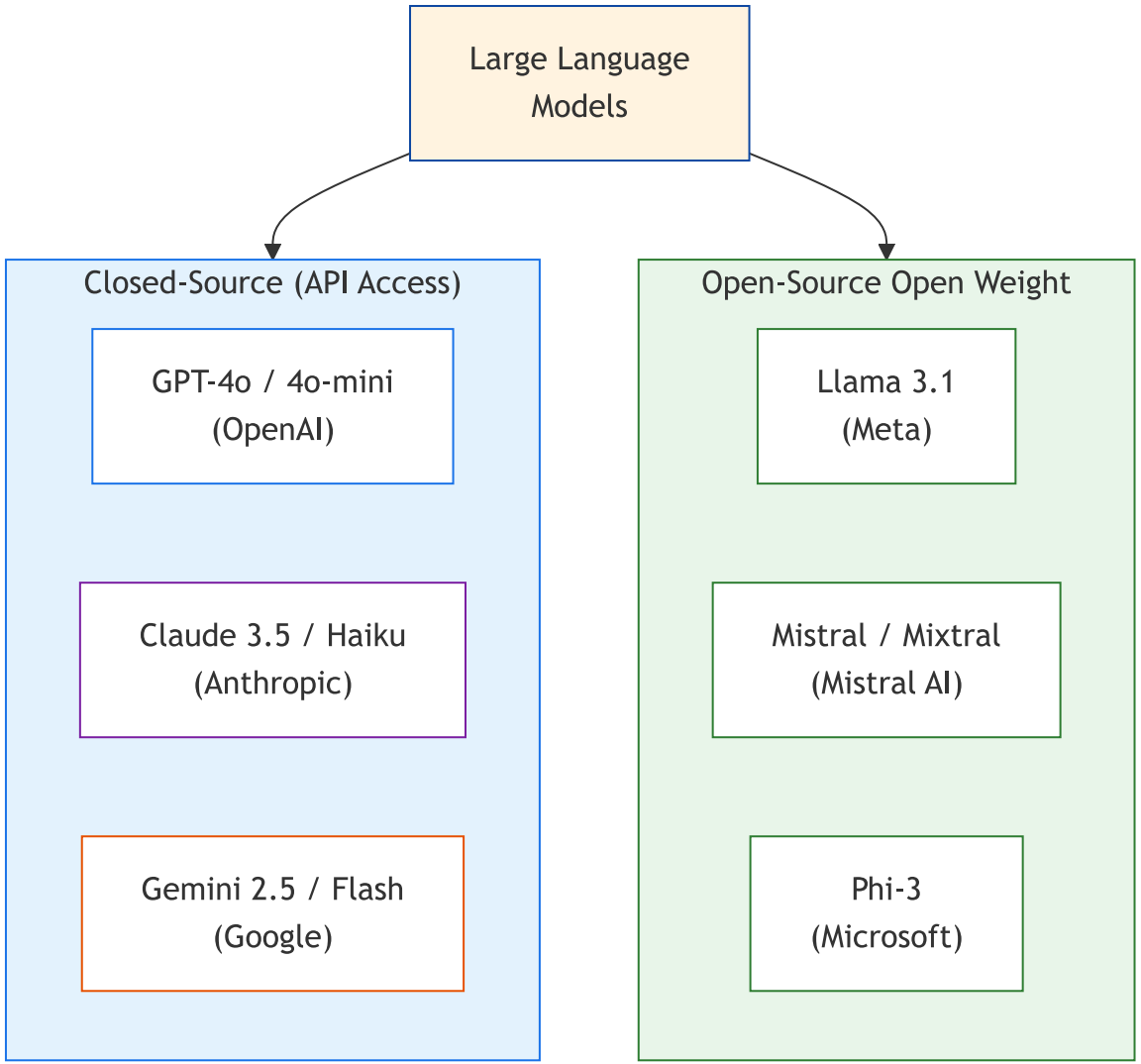
> Stretch goal: compare two prompts, or try a different small model from the hub.

The Modern LLM Landscape

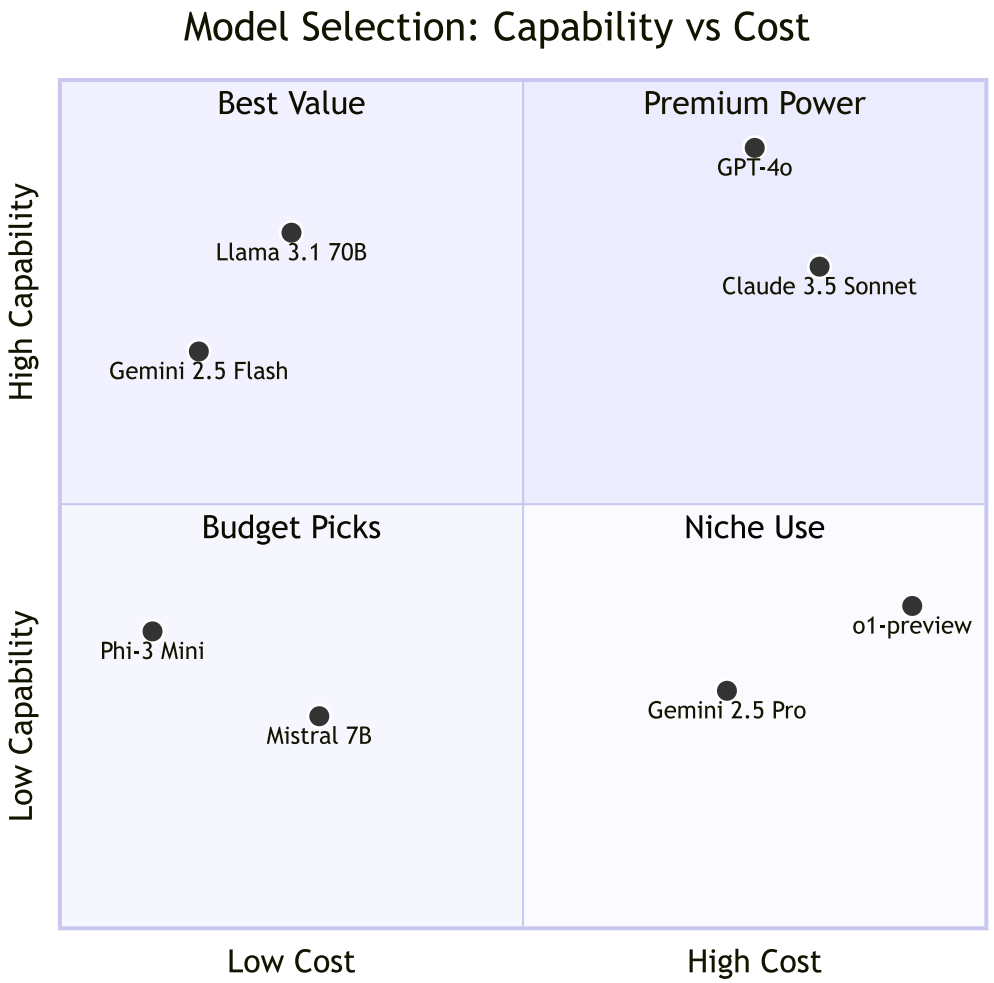
What is out there today

KavinSchool

Kinds of Models Today



Model Comparison: Capability vs Cost



Working with LLM APIs

Most production use starts with an API call:

- Send messages, get a completion back
- Same request shape across providers
- This course routes calls through a shared client so provider swaps need no code changes

You use this client in the prompting and agent labs later today.

> **TIP**
>
> Use an abstraction layer like LiteLLM to decouple your application from any single model provider.

Key Takeaways

- [x] Pre-training learns language once, then transfers to many tasks
- [x] BERT is an encoder for understanding; GPT is a decoder for generation
- [x] The Transformer's self-attention made large, scalable models possible
- [x] Generation is repeated next-token prediction from a probability distribution
- [x] Today's landscape spans closed APIs, open weights, reasoning, and multimodal

Questions?

Next: Kahoot, then lunch

Kahoot: 10 minutes | Lunch: 45 minutes

KavinSchool